SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1108

COURSE OUTCOME COVERED

CO4: Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL3)*
Assessment Tool Weightage: 30%

QUESTIONS: 15 **Total Marks:30**

Annexure A

CLASS TEST

Code of Conduct:

I E.Ragul / 192571032(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
E.Ragul

Annexure A

CLASS TEST

QUESTIONS: **Total Marks: 30**

Q1.Define architectural patterns in object-oriented systems. List any two common patterns. Explain how the Model-View-Controller (MVC) architecture improves modularity and maintainability.

[L1, L2, L3]

Q2.What is the Data Access Object (DAO) Pattern? Mention two advantages. Explain how DAO separates database-related operations from application logic.

[L2, L3, L4]

Q3.What are object-oriented design guidelines? Identify any two categories of design guidelines. Explain the role of the Liskov Substitution Principle (LSP) in developing reusable software components.

[L2, L3, L4]

Q4.Define database persistence in object-oriented applications. Mention two persistence mechanisms. Explain how an Object-Relational Mapping (ORM) framework supports persistent object management.

[L1, L2, L3]

Q5.What is an Application Layer? State two major responsibilities. Explain how the Application Layer coordinates use cases without directly implementing domain business rules.

[L2, L3, L4]

Q6.Explain the structure of a three-tier architecture consisting of Presentation, Business, and Data layers. Analyze the flow of a customer transaction through these layers in an e-commerce application.

[L2, L4, L5]

Q7.What is the role of a Business Logic Layer? List two important functions. Explain how it validates and processes requests received from the presentation layer.

[L2, L3, L4]

Q8.Explain the significance of cohesion and coupling in object-oriented design. Analyze how high cohesion and low coupling contribute to software maintainability and reusability.

[L3, L4, L5]

Q9.What is encapsulation in an object-oriented system? Explain how encapsulation protects object state and compare it with data hiding using a suitable example.

[L2, L4]

Q10.Design a layered object-oriented system for an online banking application. Identify the major layers, recommend suitable design patterns, and justify how the architecture improves security, scalability, and maintainability.

[L3, L4, L5]

Q11.What is the role of the Infrastructure Layer in object-oriented architecture? List two responsibilities. Explain how it supports technical services such as database access and external system communication.

[L1, L2, L3]

Q12.Define the Singleton Design Pattern. Mention its major characteristics. Explain its appropriate use in managing shared resources and discuss one limitation of the pattern.

[L2, L3, L4]

Q13.What is dependency injection in object-oriented design? List two benefits. Analyze how dependency injection reduces tight coupling between classes and improves testability.

[L2, L3, L4]

Q14.Explain the working of the Strategy Design Pattern. Compare the roles of Context and Strategy with a suitable real-world application example.

[L3, L4, L5]

Q15.Design a layered architecture for a Library Management System. Identify suitable layers, recommend relevant design patterns, and justify how your proposed architecture supports flexibility and maintainability.

[L3, L4, L5]

ANSWERS:

CLASS TEST – Object-Oriented System Layers, Design Principles & Patterns

Q1. Define architectural patterns in object-oriented systems. List any two common patterns. Explain how MVC improves modularity and maintainability.

Answer:
Architectural patterns are **reusable structures or templates for organizing the components of a software system** and defining how they interact.
Two common architectural patterns:

1. Model-View-Controller (MVC)
2. Layered Architecture

MVC

MVC divides an application into three components:

- **Model:** Manages data and business objects.
- **View:** Displays information to users.
- **Controller:** Handles user requests and coordinates Model and View.

```
User → Controller → Model
          ↓
         View
```

Benefits:

- Separates user interface from business logic.
- Reduces coupling between components.
- Makes individual components easier to test.
- Allows UI changes without changing business logic.
- Improves maintainability and reusability.

Q2. What is the DAO Pattern? Mention two advantages. Explain how DAO separates database operations from application logic.

Answer:
The **Data Access Object (DAO) Pattern** provides a separate object responsible for performing database operations.
Example:

```
Application Logic
        ↓
    StudentDAO
        ↓
    Database
```

Two advantages:

1. Reduces coupling between application logic and database code.
2. Improves maintainability and testability.

DAO provides methods such as:

```
save()
findById()
update()
delete()
```

The application service calls these methods instead of directly executing database operations. Therefore, database-related code remains inside the DAO, while business logic remains in service/domain classes.

Q3. What are object-oriented design guidelines? Identify any two categories. Explain the role of LSP.

Answer:
Object-oriented design guidelines are **principles and recommended practices used to create reusable, maintainable, flexible, and understandable object-oriented software**.
Two categories:

3. **Class and object design guidelines**
4. **Inheritance and polymorphism guidelines**

Liskov Substitution Principle (LSP)

LSP states that:
Objects of a subclass should be replaceable for objects of its superclass without affecting the correctness of the program.
Example:

```
          Vehicle
           /   \
         Car   Bus
```

Q4. Define database persistence. Mention two persistence mechanisms. Explain ORM.

Answer:

Database persistence is the process of storing an object's state so that it continues to exist after the application stops running.

Two persistence mechanisms:

5. Object-Oriented Database Management System (OODBMS)
6. Object-Relational Mapping (ORM)

ORM

ORM maps object-oriented classes to relational database tables.

```
Object → ORM → Database Table
```

For example:

```
Student object
      ↓
Student table
```

ORM automatically manages operations such as:

- Insert
- Update
- Retrieve
- Delete

It reduces the need to write database queries manually and provides easier persistent object management.

Q5. What is an Application Layer? State two responsibilities. Explain how it coordinates use cases.

Answer:

The **Application Layer** coordinates application-specific operations or **use cases**. It acts between the presentation layer and domain/business objects.

Two responsibilities:

1. Coordinate use cases.
2. Control the sequence of application operations.

Example:

```
User Interface
      ↓
Application Layer
      ↓
Domain Layer
      ↓
Data Access
```

For a banking application, `TransferService` may coordinate:

```
Validate account
      ↓
Check balance
```

Q6. Explain three-tier architecture and analyze a customer transaction in an e-commerce application.

Answer:
Three-tier architecture divides the system into:

1. Presentation Layer

Provides the user interface.
Examples:

- Website
- Mobile screen
- Shopping cart page

2. Business Layer

Contains application and business processing.
Examples:

- Order validation
- Price calculation
- Payment processing

3. Data Layer

Manages persistent data.
Examples:

- Customer database
- Product database
- Order database

```
Presentation Layer
        ↓
Business Layer
        ↓
Data Layer
        ↓
Database
```

Customer Transaction Flow

Suppose a customer purchases a product:

```
Customer
   ↓
Shopping Cart / Checkout
   ↓
Order Service
   ↓
Validate product and price
   ↓
Process payment
```

Q7. What is the role of a Business Logic Layer? List two functions.

Answer:
The **Business Logic Layer (BLL)** contains the rules and processing required to perform the application's business operations.
Two important functions:
1. Validate user requests.
2. Process business transactions.

Example:
```
Presentation Layer
        ↓
Business Logic Layer
        ↓
Data Access Layer
```
For a bank transfer, the BLL can:
```
Check account
     ↓
Validate balance
     ↓
Validate transfer
     ↓
Process transfer
     ↓
Save transaction
```
The BLL ensures that requests received from the presentation layer follow the required business rules before modifying data.

Q8. Explain cohesion and coupling. Analyze how high cohesion and low coupling improve maintainability.

Answer:

Cohesion

Cohesion indicates **how closely related the responsibilities within a class or module are**.

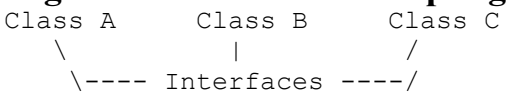
- High cohesion → one focused responsibility.
- Low cohesion → many unrelated responsibilities.

Example:
```
High Cohesion:
ReportService → only generates reports
```

Coupling

Coupling indicates **how strongly one class or module depends on another**.

High Cohesion + Low Coupling



Benefits:

- Easier maintenance.
- Easier testing.
- Components can be reused.
- Changes have less impact.
- New functionality can be added more easily.

Therefore, good object-oriented design generally aims for **high cohesion and low coupling**.

Q9. What is encapsulation? Compare it with data hiding.

Answer:
Encapsulation is the process of combining data and methods that operate on that data inside a class.
Example:

```
class BankAccount
{
private:
    double balance;

public:
    void deposit(double amount)
    {
        balance += amount;
    }

    double getBalance()
    {
        return balance;
    }
};
```

Here, `balance` and its operations are encapsulated inside `BankAccount`.

Data Hiding

Data hiding means **restricting direct access to internal data**.

```
private:
    balance
```

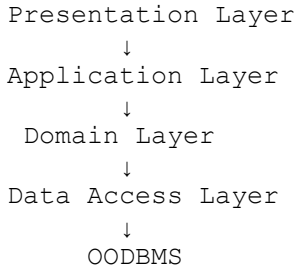
The user cannot directly modify `balance`.

Comparison

Encapsulation	Data Hiding
Combines data and methods into a class	Restricts access to internal details
Focuses on organizing objects	Focuses on protecting implementation/data

Q10. Design a layered object-oriented system for an Online Banking Application.

Answer:
A suitable architecture is:



1. Presentation Layer

Classes:
`LoginView`
`AccountView`
`TransferView`
`LoanView`

2. Application Layer

Classes:
`AccountService`
`TransferService`
`LoanService`

3. Domain Layer

Classes:
`Customer`
`Account`

4. Data Access Layer

`CustomerDAO`
`AccountDAO`
`TransactionDAO`
`LoanDAO`

Suitable Patterns

- **DAO Pattern:** Database access.
- **Factory Pattern:** Account object creation.
- **Strategy Pattern:** Different transaction validation strategies.
- **Facade Pattern:** Provides a simple interface to banking services.

Benefits

Security: Database is not directly accessed by the presentation layer.
Scalability: New banking services can be added as separate services.
Maintainability: Each layer has a specific responsibility.
Testability: Services can be tested independently of the database and UI.

Q11. What is the role of the Infrastructure Layer? List two responsibilities.

Answer:
The **Infrastructure Layer** provides technical services required by other parts of the application.
Two responsibilities:

1. Database and persistence access.
2. Communication with external systems.

Example:

```
Application/Domain
        ↓
Infrastructure Layer
   ↓           ↓
Database    External API
```

It may contain:

```
DatabaseConnection
Repository Implementation
EmailService
PaymentGateway
ExternalAPIClient
```

The Infrastructure Layer hides technical implementation details from business logic.

Q12. Define Singleton Pattern. Mention characteristics and one limitation.

Answer:
The **Singleton Design Pattern** ensures that a class has **only one instance** and provides a global access point to that instance.

Characteristics

3. Only one object is created.
4. Constructor is usually private.
5. A static method provides access to the instance.

Example:

```
Application
     ↓
Singleton Logger
     ↓
Single shared object
```

It can be useful for resources such as a **shared configuration manager or logging service** where a single coordinated instance is appropriate.

Limitation

Singleton introduces **global state**, which can make unit testing and dependency management more difficult.

Q13. What is Dependency Injection? List two benefits.

Answer:
Dependency Injection (DI) is a technique where an object's required dependencies are supplied from outside instead of being created inside the object.

Without DI

```
BankService
    ↓
new Database()
```

With DI

```
Database
    ↓
BankService
```

Example:

```
class BankService
{
private:
    Database* db;

public:
    BankService(Database* database)
    {
        db = database;
    }
};
```

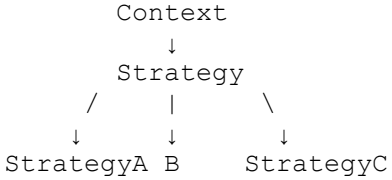
Two benefits:

6. Reduces tight coupling.
7. Improves testability.

A real database can be replaced with a fake or mock database during testing.

Q14. Explain Strategy Pattern. Compare Context and Strategy.

Answer:
The **Strategy Design Pattern** defines a family of algorithms, places each algorithm in a separate class, and allows the algorithm to be selected at runtime.



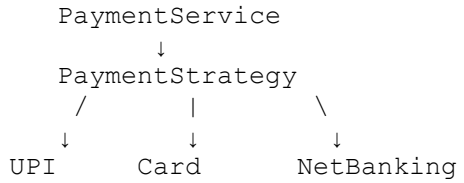
Context

The **Context** uses a strategy but does not implement the algorithm itself.

Strategy

The **Strategy** defines the common interface for different algorithms.

Real-World Example: Payment

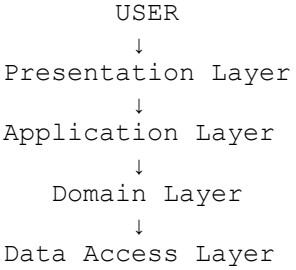


The payment service can change the payment strategy without changing its main logic.
Benefits:

- Flexible algorithm selection.
- Reduces conditional statements.
- Supports Open/Closed Principle.
- Makes individual strategies easier to test.

Q15. Design a layered architecture for a Library Management System.

Answer:
A suitable layered architecture is:



1. Presentation Layer

```
StudentView
LibrarianView
BookView
```

Responsible for displaying information and receiving user input.

2. Application Layer

```
BookService
MemberService
BorrowService
ReturnService
```

Coordinates library use cases.

3. Domain Layer

```
Book
Member
Librarian
Loan
Fine
```

Contains library-related objects and business rules.

4. Data Access Layer

```
BookDAO
MemberDAO
LoanDAO
FineDAO
```

Handles OODBMS persistence.

Suitable Design Patterns

DAO Pattern

```
BookService → BookDAO → OODBMS
```

Separates database operations from business logic.

Factory Pattern

```
BookFactory → Book objects
```

Centralizes creation of different book types.

Strategy Pattern

Can be used for different fine-calculation strategies.

```
FineStrategy
 /       \
Student   Faculty
Fine      Fine
```

Benefits

- **Flexibility:** New book types and services can be added easily.
- **Maintainability:** Each layer has a clear responsibility.
- **Reusability:** Services and domain classes can be reused.
- **Testability:** Layers can be tested independently.
- **Low coupling:** Presentation and domain logic do not directly depend on OODBMS.

RUBRICS FOR CALCULATION:

CLASS TEST

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand